

Lecture: Avalon Interface, part 1

Lecturer: Dr. Nguyen Anh Vinh

Ref: Truong Quang Vinh, IT-SoC Lab

❖ Class content

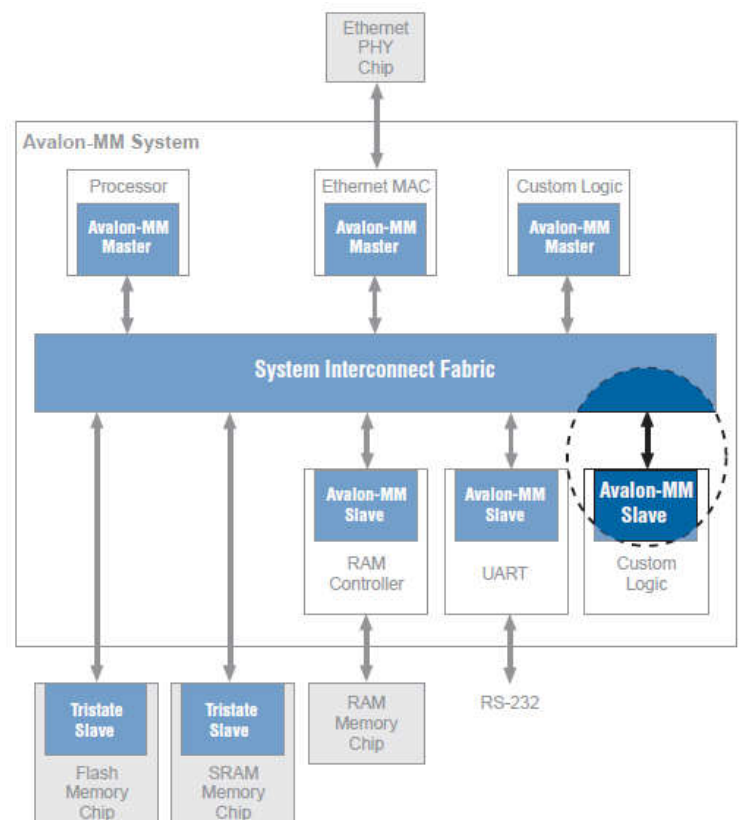
- Avalon-MM interface
- Avalon-MM tristate interface
- Avalon-ST interface
- Conduit interface

❖ Required documents

- Avalon interface specification
(http://www.altera.com/literature/manual/mnl_avalon_spec.pdf)

1. **Avalon Memory Mapped Interface (Avalon-MM)**—an address-based read/write interface typical of master-slave connections.
2. **Avalon Streaming Interface (Avalon-ST)**—an interface that supports the unidirectional flow of data, including multiplexed streams, packets, and DSP data.
3. **Avalon Memory Mapped Tristate Interface**—an address-based read/write interface to support off-chip peripherals.
4. **Avalon Clock**—an interface that drives or receives clock and reset signals to synchronize interfaces and provide reset connectivity.
5. **Avalon Interrupt**—an interface that allows components to signal events to other components.
6. **Avalon Conduit**—an interface that allows signals to be exported out at the top level of an SOPC Builder system

- ❖ Avalon Memory-Mapped (Avalon-MM) interfaces are used for read/write interfaces on master and slave components in a memory-mapped system.
- ❖ Interface between components:
 - Microprocessor
 - Memories
 - UARTs
 - Timers
- ❖ Types of Transfers:
 - Read
 - Write
 - Pipelined transfer



❖ Slave: corresponds to the control signals from Master

Table 3–1. Avalon-MM Slave Port Signals (1) (Part 1 of 4)

Signal Type	Width	Dir	Req'd	Description
Fundamental Signals				
read read_n	1	In	No	Asserted to indicate a read transfer. If present, readdata is required.
write write_n	1	In	No	Asserted to indicate a write transfer. If present, writedata is required.
address	1-32	In	No	Specifies an offset into the slave address space. Each slave address value selects a word of slave data. For example, address= 0 selects the first <slave data width> bits of slave data; address=1 selects the second <slave data width> bits of slave data.
readdata	8,16,32, 64,128, 256,512, 1024	Out	No	The readdata provided by the slave in response to a read transfer.

Table 3–1. Avalon-MM Slave Port Signals (1) (Part 2 of 4)

Signal Type	Width	Dir	Req'd	Description
writedata	8,16,32, 64,128, 256,512, 1024	In	No	Data from the system interconnect fabric for write transfers. The width must be the same as the width of readdata if both are present.
byteenable byteenable_n	1,2,4,8, 16,32,64, 128	In	No	Enables specific byte lane(s) during transfers. Each bit in byteenable corresponds to a byte in writedata and readdata. During writes, byteenables specify which bytes are being written to; other bytes should be ignored by the slave. During reads, byteenables indicates which bytes the master is reading. Slaves that simply return readdata with no side effects are free to ignore byteenables during reads. When more than one bit is asserted, all asserted lanes are adjacent. The number of adjacent lines must be a power of two, and the specified bytes must be aligned on an address boundary for the size of the data. The following values are legal for a 32-bit slave: 1111 writes full 32 bits 0011 writes lower 2 bytes 1100 writes upper 2 bytes 0001 writes byte 0 only 0010 writes byte 1 only 0100 writes byte 2 only 1000 writes byte 3 only

<code>begintransfer</code>	1	In	No	Asserted by the system interconnect fabric for the first cycle of each transfer regardless of <code>waitrequest</code> and other signals.
Wait-State Signals				
<code>waitrequest</code> <code>waitrequest_n</code>	1	Out	No	Asserted by the slave when it is unable to respond to a <code>read</code> or <code>write</code> request. When asserted, the control signals to the slave, with the exception of <code>begintransfer</code> and <code>beginbursttransfer</code> , remain constant, as is illustrated by Figure 3–7 on page 3–13 . An Avalon-MM slave may assert <code>waitrequest</code> during idle cycles. An Avalon-MM master may initiate a transaction when <code>waitrequest</code> is asserted. The design of Avalon-MM slaves must take these possibilities into account.

Table 3–1. Avalon-MM Slave Port Signals (1) (Part 3 of 4)

Signal Type	Width	Dir	Req'd	Description
<code>arbiterlock</code> <code>arbiterlock_n</code>	1	In	No	<code>arbiterlock</code> ensures that once a master wins arbitration, it maintains access to the slave for multiple transactions. It is de-asserted coincident with <code>read</code> or <code>write</code> and with the deassertion of the last locked transaction <code>read</code> or <code>write</code> signal. <code>Arbiterlock</code> assertion does not guarantee that arbitration will be won, but after the <code>arbiterlock</code>-asserting master has been granted, it retains grant until it deasserts <code>arbiterlock</code>, whether or not it is making an access. A master equipped with <code>arbiterlock</code> cannot be a burst master. Arbitration priority values for <code>arbiterlock</code>-equipped masters are ignored. <code>arbiterlock</code> is particularly useful for read-modify-write operations, where master A reads 32-bit data that has multiple bitfields, changes one field, and writes the 32-bit data back. If master B were to be able to write between Master A's read and the write, master A's write would undo what master B had done. <code>arbiterlock</code> is also for tristate-pin sharing: an SDRAM controller can use it to lock arbitration to execute an unbroken sequence of commands to an SDRAM device.

Pipeline Signals				
readdatavalid readdatavalid_n	1	Out	No	Used for variable-latency, pipelined read transfers. Asserted by the slave to indicate that the readdata signal contains valid data in response to a previous read request. A slave with readdatavalid must assert this signal for one cycle for each read access it has received. There must be at least one cycle of latency between acceptance of the read and assertion of readdatavalid. Figure 3–5 on page 3–10 illustrates the readdatavalid signal.
Burst Signals				
burstcount	1-11	In	No	During the first cycle of a burst, burstcount indicates the number of transfers the burst contains. The value of the maximum burstcount parameter must be a power of 2, so a burstcount port of width $\langle n \rangle$ can encode a max burst of size $2^{(\langle n \rangle - 1)}$. For example, a 4-bit burstcount signal can support a maximum burst count of 8. The minimum burstcount is 1.
beginbursttransfer	1	In	No	Asserted for the first cycle of a burst to indicate when a burst transfer is starting. This signal is deasserted after one cycle regardless of the value of waitrequest. Refer to Figure 3–7 on page 3–13 for an example of its use.
Flow Control Signals				
readyfordata	1	Out	No	Used for transfers with flow control. Indicates that the component is ready for a write transfer.
dataavailable	1	Out	No	Used for transfers with flow control. Indicates that the component is ready for a read transfer.

Table 3–2. Avalon-MM Slave Interface Properties (Part 1 of 2)

Name	Default Value	Legal Values	Description
readLatency	0	0–63	Read latency for fixed-latency slaves. Not used on interfaces that include the readdatavalid signal. Refer to Figure 5–5 on page 5–9 for an timing diagram that uses this property.
timingUnits	cycles	cycles, nanoseconds	Specifies the units for setupTime, holdTime, writeWaitTime and readWaitTime. Use cycles for synchronous devices and nanoseconds (depending on the timingUnits parameter) for asynchronous devices. Almost all Avalon-MM slave devices are synchronous. One example of a device that requires asynchronous timing is an Avalon-MM slave that reads and writes an off-chip bidirectional port. That off-chip device might have a fixed settling time for bus turnaround.
writeWaitTime	0	0–1000 cycles	For slave interfaces that do not use the waitrequest signal, writeWaitTime indicates the number of cycles or nanoseconds (depending on the timingUnits parameter) before the slave accepts a write. The timing is as if the slave asserted waitrequest for writeWaitTime cycles or nanoseconds. Refer to Figure 5–4 on page 5–8 for a timing diagram that uses this property.

readWaitTime	1	0–1000 cycles	For slave interfaces that don't use the waitrequest signal, readWaitTime indicates the number of cycles or nanoseconds before the slave accepts a read command. The timing is as if the slave asserted waitrequest for readWaitTime cycles.
holdTime	0	0–1000 cycles	Specifies time in timingUnits between the deassertion of write and the deassertion of chipselect, address, and data. (Only applies to write transactions.)

Table 3–2. Avalon-MM Slave Interface Properties (Part 2 of 2)

Name	Default Value	Legal Values	Description
setupTime	0	0–1000 cycles	Specifies time in timingUnits between the assertion of chipselect, address, and data and assertion of read or write.
maximumPendingRead Transactions	1 (1)	1–64	The maximum number of pending reads which can be queued up by the slave. Refer to Figure 3–5 on page 3–10 for a timing diagram that uses this property.

burstOnBurstBoundariesOnly	false	true,false	If true, burst transfers presented to this interface are guaranteed to begin at addresses which are multiples of the burst size.
linewrapBursts	false	true,false	If true, indicates that the slave implements a line wrapping burst instead of an incrementing burst. With a wrapping burst, when the address reaches a burst boundary, it wraps back to the previous burst boundary such that only the low order bits need to be used for addressing. To address 0xC, a wrapping burst with burst boundaries every 32 bytes across a 32-bit interface would write to addresses 0xC, 0x10, 0x14, 0x18, 0x1C, 0x0, 0x4, and 0x8.
maxBurstSize	1	64	The maximum burst size that a slave can accept.
bridgesToMaster	null	Avalon-MM master on the same component	An Avalon-MM bridge consists of a slave and a master, and has the property that an access to the slave requesting a particular byte or bytes will cause the same byte or bytes to be requested by the master.
associatedClock	—	—	Name of the clock interface that this Avalon-MM slave interface is synchronous to.

- ❖ This section describes a typical Avalon-MM slave that supports read and write transfers with slave-controlled waitrequest.
- ❖ The slave can stall the system interconnect fabric for as many cycles as required by asserting the waitrequest signal.
- ❖ If a slave uses waitrequest for either read or write transfers, it must use waitrequest for both.

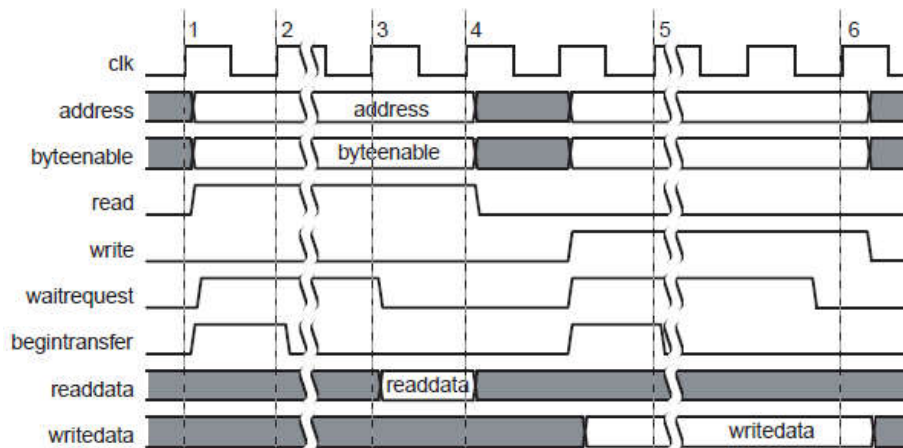


Figure 3-3. Slave Read and Write Transfers with Waitrequest

1. The master asserts address and read on the rising edge of clk.
2. The next rising edge of clk marks the end of the first and only wait-state cycle because the readWaitTime is 1.
3. The slave captures readdata on the rising edge of clk, and the read transfer ends.
4. writedata, address, byteenable, and write signals are available to the slave.
5. Because writeWaitTime is 2, the transfer terminates after completing. The data and control signals are held constant until this time.

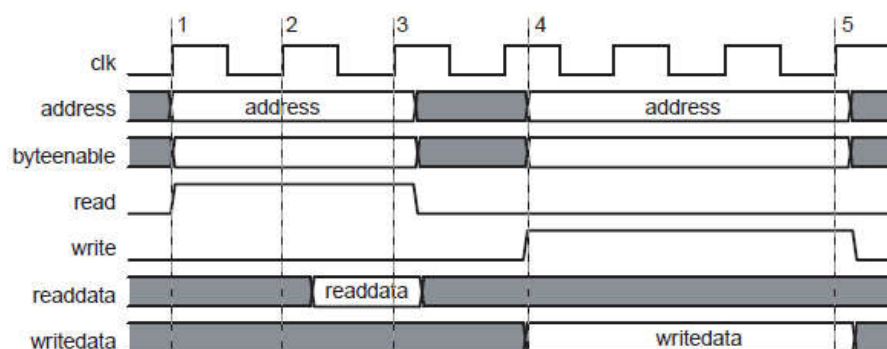
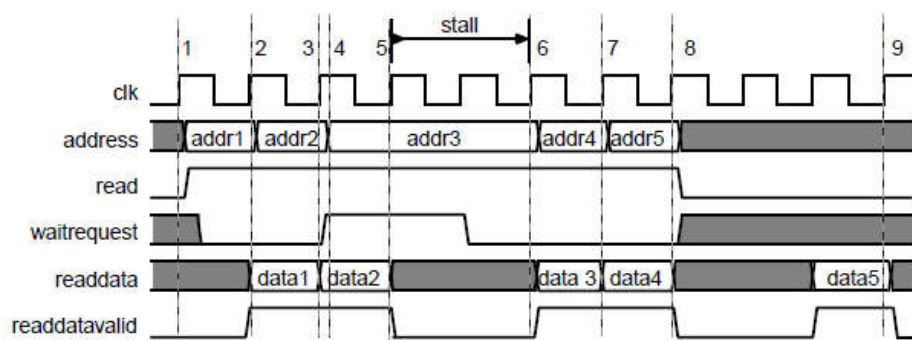


Figure 3-4. Slave Read and Write Transfer with Fixed Wait-States

1. The master asserts address and read, initiating a read transfer.
2. The slave captures addr1, and immediately provides the response data1 and asserts readdatavalid.
3. The slave captures addr2 and immediately provides the response data2 and asserts readdatavalid. System interconnect fabric captures data1.
4. The slave asserts waitrequest for two cycles causing the third transfer to be stalled.
5. System interconnect fabric captures data2.
6. The slave drives readdatavalid and valid readdata in response to the third read transfer.
7. The data from transfer 3 is captured by the interconnect at the same time that addr4 is captured by the slave.
8. The slave captures addr5. System interconnect fabric captures data4.
9. data5 is presented with readdatavalid completing the data phase for the final pending read transfer.



IT-SoC Lab

1. The master asserts address, burstcount, write, and drives the first unit of writedata. The slave immediately asserts waitrequest, indicating that it is not ready to proceed with the transfer.
2. waitrequest is low; the slave captures addr1, burstcount, and the first unit of writedata. On subsequent cycles of the transfer, address and burstcount are ignored.
3. The slave port captures the second unit of data at the rising edge of clk.
4. The burst is paused while write is deasserted.
5. The slave captures the third unit of data at the rising edge of clk.
6. The slave asserts waitrequest. In response, all outputs are held constant through another clock cycle.
7. The slave captures the last unit of data on this rising edge of clk. The slave write burst ends.

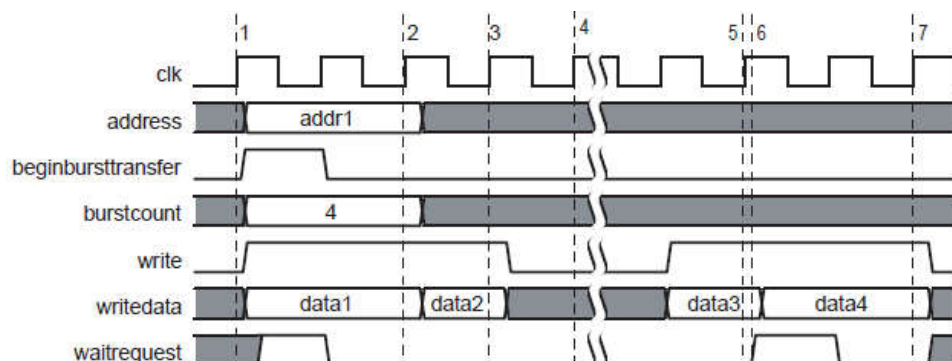
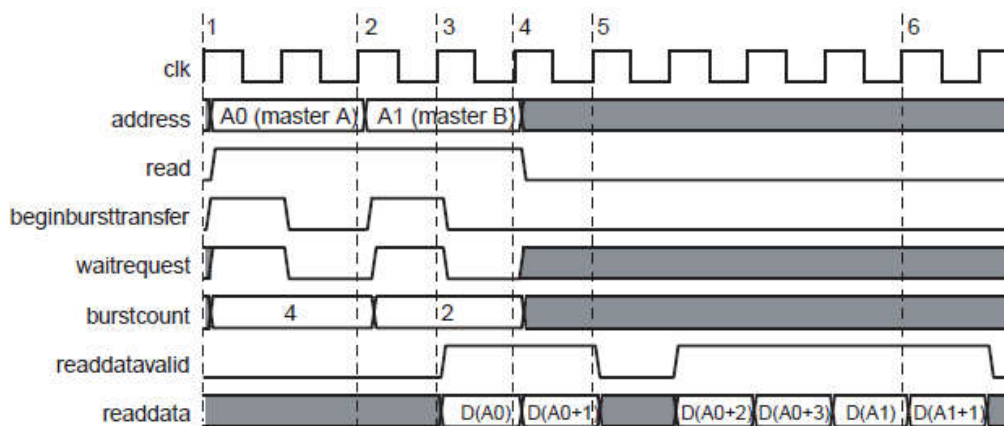


Figure 3-7. Slave Write Burst

IT-SoC Lab

1. Master A asserts address (A0), burstcount, and read after the rising edge of clk.
2. The slave captures A0 and burstcount at this rising edge of clk.
3. Master B drives address (A1), burstcount, and read. The slave asserts waitrequest, causing all inputs except beginbursttransfer to be held constant.
4. The slave presents valid readdata and asserts readdatavalid, transferring the first word of data for master A.
5. The second word for master A is transferred. The slave deasserts readdatavalid pausing the read burst.
6. The first word for master B is returned.



❖ Avalon-MM Master:

- Allow to use bursts for both read and write
- Ability to read with multiple slave peripherals

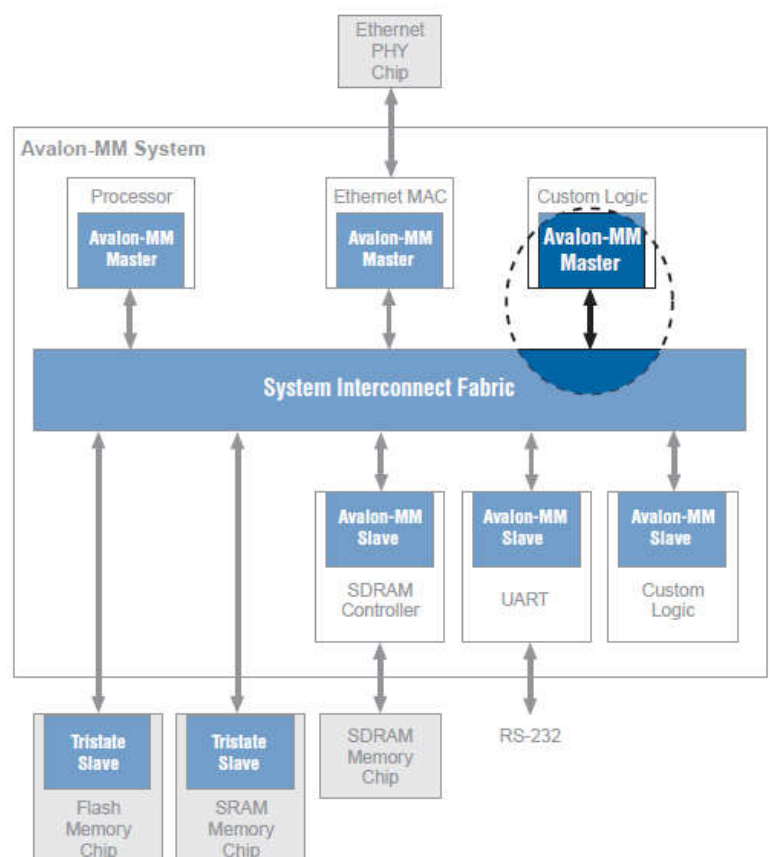


Table 3–4. Avalon-MM Master Signals (Part 1 of 3)

Signal Type	Width	Direction	Req'd	Description
Fundamental Signals				
address	1-32	Out	Yes	The address signal represents a byte address regardless of the data-width of the master. The value of the address must be aligned to the data width. To write to specific bytes within a data word, the master must use the <code>byteenable</code> signal. Masters always issue byte addresses, regardless of the data width of the master or slave port. The system interconnect fabric translates this address into a word address in the slave's address space so that each slave access is for a word of data from the perspective of the slave.
read read_n	1	Out	No	Read request signal from the master. Not required if the master never performs read transfers. If present, <code>readdata</code> must also be present.
readdata	8,16,32,64, 128, 256, 512, 1024	In	No	Data signal for read transfers.
write write_n	1	Out	No	Write request signal from the master. Not required if the master never performs write transfers. If present, <code>writedata</code> must also be used.
writedata	8,16,32,64, 128, 256, 512, 1024	Out	No	Data signal from the master for write transfers. Not required if the master never performs write transfers. If <code>readdata</code> is also present, <code>readdata</code> and <code>writedata</code> must be the same width.

IT-SoC Lab

writedata	8,16,32,64, 128, 256, 512, 1024	Out	No	Data signal from the master for write transfers. Not required if the master never performs write transfers. If <code>readdata</code> is also present, <code>readdata</code> and <code>writedata</code> must be the same width.
byteenable byteenable_n	1, 2,4,8, 16, 32, 64, 128	Out	No	Enables specific byte lanes during transfers on ports of width greater than 8 bits. Each bit in <code>byteenable</code> corresponds to a byte lane in <code>writedata</code> and <code>readdata</code> . The master bit <code><n></code> of <code>byteenable</code> indicates whether byte <code><n></code> is being written to. During writes, <code>byteenables</code> specify which bytes to write. Other bytes should be ignored by the slave. During reads, <code>byteenables</code> indicates which bytes the master is reading. When more than one byte lane is asserted, all asserted lanes must be adjacent. The number of adjacent lines must be a power of 2, and the specified bytes must be aligned on an address boundary for the size of the data. The are legal values for a 32-bit slave: 1111 writes full 32 bits 0011 writes lower 2 bytes 1100 writes upper 2 bytes 0001 writes byte 0 only 0010 writes byte 1 only 0100 writes byte 2 only 1000 writes byte 3 only

IT-SoC Lab

Table 3–4. Avalon-MM Master Signals (Part 2 of 3)

Signal Type	Width	Direction	Req'd	Description
waitrequest waitrequest_n	1	In	Yes	Forces the master to wait until the system interconnect fabric is ready to proceed with the transfer. At the start of all transfers, a master initiates the transfer, and waits until waitrequest is deasserted. Masters must keep its control signals the same on subsequent cycles if waitrequest is asserted
arbiterlock arbiterlock_n	1	Out	No	arbiterlock ensures that once a master wins arbitration, it maintains access to the slave for multiple transfers. It is de-asserted coincident with read or write and with the deassertion of the last locked transfer read or write signal. Arbiterlock assertion does not guarantee that arbitration will be won, but after the arbiterlock-asserting master has been granted, it retains grant until it deasserts arbiterlock, whether or not it is making an access. Burst masters cannot use the arbiterlock signal. Arbitration priority values for arbiterlock-equipped masters are ignored.

IT-SoC Lab

Pipeline Signals

readdatavalid readdatavalid_n	1	In	No	For pipelined read transfers with latency. Indicates that valid data is present on the readdata lines. Required if the master supports pipelined reads. Bursting masters with read functionality must include the readdatavalid signal.
Burst Signals				
burstcount	1-11	Out	No	Used by bursting masters to indicate the number of transfers in each burst. The minimum value for burstcount is 1. For a burstcount signal of width $\langle n \rangle$, the maximum burst length is $2^{\langle n \rangle - 1}$. Bursting masters with read functionality must include the readdatavalid signal.

IT-SoC Lab

Table 3–4. Avalon-MM Master Signals (Part 3 of 3)

Signal Type	Width	Direction	Req'd	Description
Reset Signals				
resetrequest resetrequest_n	1	Out	No	Asserted by the master to request a reset the entire system.

Note to Table 3–4:

(1) All Avalon signals are active high. Avalon signals that can also be asserted low list an `_n` version of the signal in the **Signal Type** column.

Table 3–5. Avalon-MM Master Interface Properties

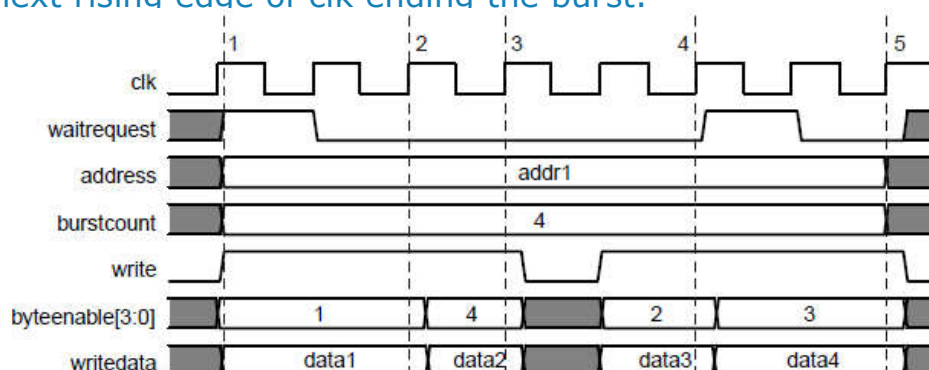
Name	Default Value	Legal Values	Description
burstOnBurstBoundariesOnly	false	true,false	If true, the master guarantees that all bursts begin on a multiple of the burst size.
linewrapBursts	false	true,false	Some memory devices implement a wrapping burst instead of an incrementing burst. The difference between the two is that with a wrapping burst, when the address reaches a burst boundary, the address wraps back to the previous burst boundary such that only the low order bits need to be used for address counting. A wrapping burst with burst boundaries every 32 bytes across a 32-bit interface to address 0xC would write to addresses 0xC, 0x10, 0x14, 0x18, 0x1C, 0x0, 0x4, and 0x8.
maxBurstSize	1	1–64	The maximum burst size that a master can send.
doStreamReads	false	true,false	Indicates that the master wishes to be held off with the <code>waitrequest</code> signal whenever it reads from a slave that has <code>dataavailable</code> deasserted.
doStreamWrites	false	true,false	Indicates that the master wishes to be held off with the <code>waitrequest</code> signal whenever it writes to a slave that has <code>readyfordata</code> deasserted.

1. The master initiates a read by asserting address, byteenable, and read.
2. The master captures readdata and deasserts read, ending the transfer.
3. waitrequest is not asserted at the rising edge of clk, so the write transfer completes.
4. The master asserts valid address, byteenable, writedata, and write beginning a second write transfer. waitrequest is asserted, so the master holds all outputs.
5. waitrequest is not asserted so the write transfer completes. The master asserts address, byteenable, and read for the next transfer. waitrequest is asserted.
6. waitrequest is not asserted at the rising edge of clk, so the read transfer completes.



IT-SoC Lab

- ❖ The master begins a burst of 4 transfers. waitrequest is asserted, pausing the burst and causing the master to hold all outputs constant.
- ❖ Because waitrequest is deasserted, the system interconnect fabric accepts the first write transfer.
- ❖ The second writedata (data2) is accepted. The master then deasserts write, pausing the burst.
- ❖ The system interconnect captures writedata (data3) and then the master presents the last unit of writedata (data4) waitrequest pauses the burst again.
- ❖ waitrequest is deasserted and the last unit of writedata (data4) is captured on the next rising edge of clk ending the burst.



IT-SoC Lab